ТЕХНИЧЕСКОЕ ЗАДАНИЕ

Эмулятор Siemens/Infineon C166 classic для динамического Reverse Engeniring

Инструмент для динамического реверс инжениринга программ, написанных для Siemens/Infineon C166 classic

Версия	1.1
Статус	Черновик
Дата	2026-10-08
Ментор	Кутаев Кирилл
Организация	MIPT x PT

Содержание

1 Общие сведения 3
1.1 Назначение документа 3
1.2 Термины и определения 3
2 Задачи и цели 4
2.1 Цель 4
2.2 Задач 4
3 Архитектура 5
3.1 Общее представление 5
3.2 Описание всех компонентов 5
3.2.1 1. Registers 5
3.2.2 2. Program Status Word 5
3.2.3 3. Arithmetic Logic Unit (ALU) 6
3.2.4 4. BitProcessor 6
3.2.5 5. Stack + BranchUnit 7
3.2.6 6. Memory 8
4 Ограничения 9
4.1 Обязательно 9
4.2 Желательно 9
5 Технологический стек 10
6 Календарный план 11

Общие сведения

Назначение документа

TODO: вода про важность тз

> **О документе**
>
> *Кратко описано что фиксирует документ*

Термины и определения

Термин	Определение
ISA	Сокращение наименования Siemens/Infineon C166 classic
BYTE	Один байт памяти
WORD	Два байта памяти

Задачи и цели

Цель

Цель проекта — разработать программный эмулятор процессорного ядра Siemens/Infineon C166 classic, предназначенный для динамического анализа и исследования исполняемого кода.

В рамках первой версии реализуется эмулятор, умеющий работать с регистрами, арифметической и логическими операциями, адресацией памяти, управлением потоком исполнения и операциями со стеком.

Задач

1. **Исследовать архитектуру ISA**

   Изучить документацию на ISA. Требуется определить состав регистров, флагов и изучить работу с памятью. Установить перечень оддерживаемых инструкций. Отдельного внимания заслуживает работа с 24-битной физической памятью при помощи DPP

2. **Реализация компонентов**

   Реализация основных компонентов эмулятора. Про каждый из них написано подробнее в разделе **Архитектура**.

3. **Тестирование**

   Каждый компонент должен быть протестирован отдельно. Так же нужно протестировать работу эмулятора в целом используя готовые бинарные файлы с известным поведением.

4. **Документирование**

   Написание документации по проекту и инструкций о ипользованию эмулятора.

Архитектура

Общее представление

Цикл ядра представляет следующую цепочка событий:

$$\text{Fetch} \rightarrow \text{Decode} \rightarrow \text{Execute} \rightarrow \text{WriteBack}$$

Во время выполнения цикла эти четыре компонента могут обращаться к другим, например:

- Registers
- PSW
- Memory
- ALU
- BitProcessor
- Stack
- BranchUnit

(взаимодействие между компонентами пока в заслопленой фотке `arch.png`)

Описание всех компонентов

1. Registers

Содержит в себе все регистры процессора и предоставляет интерфесы для взаимодействия с ними:

Регистры

R0 - R15	Регистры общего назначения
Stack Pointer	Указатель на стек
Code Segment Pointer	Указатель на начало сегмента кода
Instruction Pointer	Адрес инстркуции в текущем сегменте кода
Data Page Pointer 0 - 4	Указатель на страницу памяти
Context Pointer	Адресация текущего конспекта
Program Status Word	Флаги состояния процессора
MDH и MDL (вместе MD)	Регистры для умножения и деления (32 бита)

Для регистров R0-R7 есть возможность адресовываться отдельно к старшему и бладшему байту (RH0-RH7 - старший байт; RL0-RL7 - младший байт). Копмпонент должен предоставлять интерфейс для записи / чтения значений в регистры.

2. Program Status Word

Содержит в себе флаги процессора. Компонент предоставляет доступ к чтению / записи любого флага из этого регистра.

Флаги

N (negative)	Флаг знака (1 - если результат ALU отрицателен)

C (carry)	Флаг беззнакового переполнения
V (overflow)	Флаг знакового переполнения
Z (zero)	Флаг нуля (1 - если результат ALU равен 0)
USR0	Пользовательский флаг

3. Arithmetic Logic Unit (ALU)

Компонент для арифметики. Должен уметь выполнять базовые арифметические операции, возвращать результат и изменение флагов

Операции ALU

ADD	Сложение
ADDB	Сложение (только младший байт)
ADDC	Сложение с учетом флага C от предыдущей операции
ADDCB	Сложение с учетом флага C от предыдущей операции (только младший байт)
SUB	Вычитание
SUBB	Вычитание (только младший байт)
SUBC	Вычитание с учетом флага C от предыдущей операции
SUBCB	Вычитание с учетом флага C от предыдущей операции (только младший байт)
MUL	Умножение
MULU	Беззнаковое умножение
DIV	Деление
DIVL	Деление (32 бит)
DIVU	Беззнаковое деление
DIVLU	Беззнаковое деление (32 бит)
CMP	Сравнение операндов (изменяет только флаги)
CMPB	Сравнение операндов (только младший байт) (изменяет только флаги)
NEG	Арифметическое отрицание

4. BitProcessor

Компонент аналогичный ALU, который предоставляет перации с битами

Операции BitProcessor

AND	Побитовое И
OR	Побитовое ИЛИ
XOR	Побитовое исключающее ИЛИ
CPL	Побитовое НЕ
SHL	Сдвиг влево
SHR	Сдвиг вправо
ASHR	Арифметический сдвиг вправо (сохраняется знаковый бит)
ROL	Циклический сдвиг влево
ROR	Циклический сдвиг вправо

5. Stack + BranchUnit

Компонент, эмулирующий работу стека. Содержит в себе Stack Pointer (SP), механизмы контроля переполнения и опустошения стека (STKOV и STKUN), интерфейсы для реализации вызова функции (CALL) и возрата (RET). Ветвление реализуется с помощью условных вызовов подпрограмм.

Операции Stack

PUSH	Уменьшает SP на 2 и записывает WORD на стек
POP	Читает WORD со стека и увеличивает SP на 2
CALL	Сохраняет адрес возврата на стеке и передает управление функции
RET	Читает адрес возврата и передает управелние по нему
RETS	Читает адрес возврата и сегмент и передает управелние по ним
RETP	Читает адрес возврата и сохраненный регистр и передает управелние по нему, восстанавливая значение регистра
STKOV	Регистр для контроля переполнения стека
STKUN	Регистр для контроля пустоты стека
CALLA	Если выполняется условие, передать управление функции (абсолютный адрес)
CALLI	Если выполняется условие, передать управление функции (адрес функции берется из регистра)
CALLR	Безусловно передать управление функции (относительный адрес)
CALLS	Безусловно передать управление функции и другого сегмента
PCALL	Передать управление функции, сохранив определенный регистр на стеке (восстановится после оператора RETP)

JMPA	Условный переход по абсалютному адрес
JMPI	Условный переход по адресу из регистра
JMPR	Условный переход по относительному адресу
JMPS	Безусловный переход по бсалютному адресу с учетом сегмента
JB	Условный переход, если установлен определнный бит
JNB	Условный переход, если НЕ установлен определнный бит
JBC	Условный переход, если установлен определнный бит (бит сбрасывается)
JNBC	Условный переход, если НЕ установлен определнный бит (бит устанавливается)

6. Memory

Это компонент должен предоставлять интерфейс для чтения и записи данных и учитывать механизмы DPP.

Операции Memory

read8	Прочитать BYTE по адресу
read16	Прочитать WORD по адресу
write8	Записать BYTE по адресу
write16	Записсать WORD по адресу
LogicToPhysic	Перевести 16-битный логический адрес в 24-битный физический учитывая механизм DPP
PhysicToLogic	Перевести 24-битный физический адрес в 16-битный логический учитывая механизм DPP

Ограничения

В силу ограниченности по времени предлагается выделить два блока: обязательное и желательное. Проект считается завершенным, если реализавано все из блока *обязательно*.

Обязательно

1. Загрузка бинарного файла в память.
2. Установка начального состояния процессора.
3. Запуск исполнения.
4. Пошаговое исполнение одной инструкции.
5. Просмотр текущих значений регистров и PSW.
6. Просмотр содержимого памяти.
7. Получение информации о текущей инструкции.
8. Остановка исполнения при достижении заданного адреса.
9. Остановка при неподдерживаемой инструкции или другой определённой ошибке исполнения.
11. Получение конечного состояния процессора после завершения тестовой программы.

Желательно

1. Точки останова.
2. Трассировка исполнения.
3. Журнал изменений регистров.
4. Журнал обращений к памяти.
5. Сохранение и загрузка состояния эмулятора.
6. Экспорт трассировки для последующего анализа.

Технологический стек

C++ 20	Важна интеграция с IDA PRO, скорость работы и ручное управление памятью *(Core)*
Python 3.14	Для прототипов и верхних уровней логики

* Все еще обсуждается

Календарный план

№	Этап / работа	Начало	Окончание
1	Написание документации, изучения ISA	28.09.26	11.10.26
2	Реализуются следующие компоненты: Registers, PSW, ALU + их тестирование и документирование	12.10.26	25.10.26
3	Реализуются следующие компоненты: BranchUnit, Stack, BitProcessor + их тестирование и документирование	26.10.26	08.11.26
4	Компонент Memory + его тестирование и документирование	09.11.26	22.11.26
5	Связка компонентов, сборка эмулятора + тестирование и документирование	23.11.26	29.11.26
6	Создание презентации и написание доклада. Подготовка к защите	30.11.26	15.12.26